Scenario 1

Scenario 1:

You are using ChatGPT API to create a customer support chatbot. Users often ask for the latest offers, but the chatbot gives old information.

Question:

- How can you fix this issue?

Logic:

- Use RAG to fetch real-time data.
- Connect to a live database for updated offers.
- Provide the chatbot with the latest information using context.

Scenario 2:

A chatbot is recommending random products even though users give clear preferences.

Question:

- How will you improve the chatbot's recommendations?

Logic:

- Ensure the chatbot uses a retriever to find the most relevant products.
- Check if the vector database has correct and updated product data.
- Fine-tune the chatbot for better understanding of user preferences.

Scenario 3:

A finance chatbot using ChatGPT API sometimes gives different answers for the same question.

Question:

- What can you do to make the chatbot more consistent?

Logic:

- Set a low temperature (e.g., 0.2) for stable answers.
- Limit random responses by using a low top_p value.
- Provide clear and specific prompts for better control.

Scenario 4:

A legal chatbot answers general questions well but struggles with specific case-related queries.

Question:

- How can you make it give more accurate answers?

Logic:

- Use RAG to fetch case laws and legal documents.
- Ensure the retriever searches legal databases using proper keywords.
- Provide the relevant legal information as context to ChatGPT.

Scenario 5:

A healthcare chatbot sometimes gives incorrect medical advice.

Question:

- How will you ensure safe and accurate responses?

Logic:

- Connect the chatbot to a verified medical knowledge base.
- Use RAG to retrieve correct medical information.
- Apply a low temperature to reduce random or creative answers.
- Ensure critical advice is reviewed by medical professionals.